

Documentação do Site – TEP

Tópicos para facilitar vcs de se achar :)

- Conexão com banco	02
- Jornada das rotas	02
- Metodos HTTP.....	03
- 3 CRUDs	03
- O que é subquery	03
- SELECTs	04
- Responsividade	05
- Criptografia de senha	05

Conexão com o banco:

No arquivo .env temos as nossas variáveis de ambiente que servem para se comunicar com o banco de dados. Quando rodamos o projeto o app.js é o primeiro a iniciar e lê o .env e sobe o servidor na porta 8080, e monta as principais rotas como /empresas, /usuários e /dashboard e vai direcionando cada solicitação para o lugar certo.

A conexão com o banco acontece no config.js que fica na pasta database esse arquivo vai montar a configuração do MYSQL com base nas variáveis do .env e usa a função executar(), que recebe uma instrução SQL, abre a conexão e executa a query, traz os resultados e fecha a conexão depois disso vai para a models.

Jornada das rotas:

Quando você preenche os dados da empresa e clica em cadastrar o js usa a função fetch() para enviar os dados usando o método post, vai para o arquivo app.js e vê as rotas, a partir dessa rota ele segue o seu “caminho” para as rotas específicas no caso de cadastro seria a empresas.js na pasta de routes, dentro do arquivo encontra a rota de cadastrar e vai para o controller que vai fazer as validações se tudo passar nas validações ele chama o model que tem as receitas com instruções específicas para o mysql, a model chama a função database.executar que passa essas instruções para o config.js que pega as instruções e abre uma conexão com o banco de dados usando as variáveis envia essas instruções e guarda os dados e retorna com a resposta que faz esse caminho inverso: do database volta para o model, do model para o controller, do controller para a rota, da rota para o [app.js](#) e do [app.js](#) para o navegador.

As rotas são os caminhos que o navegador usa para dizer o que fazer.

Os controllers recebem os pedidos vindos das rotas, depois de validar tudo e chamam a model.

As models funcionam como receitas, cada função contém instruções específicas para o banco de dados e chama o database para executar.

Resumo da jornada:

Eu cliquei -> fetch envia -> app.js encaminha -> routes identifica -> controller valida -> model monta o sql -> database executa -> mysql salva -> database retorna -> model retorna -> controller envia resposta -> rota repassa -> app.js devolve pro navegador.

Metodos HTTP:

Resumo básico caso alguém não lembre:

São verbos que definem a ação que uma requisição cliente deve executar em um recurso de um servidor.

POST: envia dados

GET: pega os dados:

PUT: substitui um recurso já existente por um novo, pode criar tbm mas usamos mais o post.

DELETE: deleta :)

3 CRUDs:

CRUD 1: Empresas

Cadastra as empresas, busca elas e retorna os dados dessa empresa e é vinculada aos seus usuários funcionários.

CRUD 2: Usuários

Cria usuários, lista os usuários de uma empresa, atualiza os cargos, e deleta eles.

CRUD 3: Níveis de Acesso

Cria os cargos junto ao usuário, aparece na listagem e é atualizado pelos usuários administradores.

O que é subquery:

É um select dentro de outro select, como se eu tivesse fazendo uma pergunta dentro de outra pergunta. Muito útil para buscar dados dinâmicos como o ano mais recente, ou fazer comparações. No site uso para pegar qual o setor que ta mais forte, pegando qual é o setor econômico que mais contribui para o município no ano mais recente, somando os valores adicionados por setor, ordenando e pegando o maior.

SELECTs:

Dos Usuarios:

Temos dois SELECTs principais: o primeiro é usado na função de autenticação que busca os dados do usuário através de um JOIN triplo entre as tabelas User, Cargo e Empresas filtrando por email e senha criptografada com SHA2, retornando id, nome, email, empresa, CNPJ e se é administrador; o segundo SELECT lista todos os funcionários de uma empresa específica através de JOIN entre User e Cargo, filtrando pela empresa e ordenando por cargo (administradores primeiro) e depois por nome em ordem alfabética, retornando id, nome, email e tipo de usuário.

Das empresas:

Temos quatro SELECTs: o primeiro busca uma empresa específica por ID retornando todos os campos da tabela Empresas; o segundo lista todas as empresas com JOIN em Municípios para trazer também o nome da cidade; o terceiro lista todos os municípios disponíveis com JOIN em Regiões para mostrar a sigla da região ordenado alfabeticamente por nome do município; e o quarto busca empresa por CNPJ usado para validação de duplicidade.

Da dash:

Temos sete SELECTs: o primeiro busca dados resumidos do dashboard com múltiplos JOINS entre Municípios, Regiões, Metricas_do_pib e Indicadores, usando subqueries para encontrar o setor com maior participação e ano mais recente; o segundo busca indicadores econômicos de um ano específico com JOIN entre Indicadores, Setores e Metricas_do_pib ordenado por valor adicionado; o terceiro lista todos os anos disponíveis na tabela Indicadores para aquele município; o quarto busca histórico completo de todos os anos com JOIN triplo incluindo informações de PIB e impostos; o quinto busca dados consolidados de um ano específico com subqueries aninhadas para calcular setor destaque e participação total no PIB; o sexto calcula arrecadação agrupada por setor com percentual de participação usando COALESCE(é uma função que retorna o primeiro valor não nulo de uma lista de expressões) e NULLIF para evitar divisão por zero; e o sétimo calcula crescimento anual do PIB per capita comparando ano atual com anterior através de self-join na tabela Metricas_do_pib, calculando tanto crescimento percentual quanto absoluto com funções ROUND e CASE para tratamento de valores nulos, ordenando pelos anos mais recentes e limitando aos últimos 10 registros.

Responsividade:

Uso media queries no css que são como regras condicionais, como se dissesse se a tela for menor que x pixels faça isso, porque assim o site se adapta a delas de diferentes tamanhos. O layout altera colunas, espaçamentos, larguras e posições conforme o tamanho da tela. O primeiro breakpoint acontece em 900px, reorganizando o layout para exibir os elementos em coluna, removendo bordas laterais, ajustando a largura para 98% e reposicionando a imagem do tepinho para dentro do fluxo da página. Em 600px, ocorre uma adaptação maior: o header passa a ficar em coluna, os inputs ficam mais compactos e a imagem reduz ainda mais o tamanho. Em telas menores que 480px, os ajustes são mais finos, reduzindo paddings, diminuindo a logo e adaptando textos.

A propriedade box-sizing: border-box impede que elementos ultrapassem o limite do container. Unidades relativas como vw tornam larguras flexíveis. O uso de flexbox reorganiza o header e outros elementos conforme o espaço. As animações continuam funcionais porque estão definidas fora dos media queries. A meta tag viewport garante que o layout seja exibido corretamente no celular sem zoom automático.

O uso de overflow: visible no container principal permite que a imagem do tepinho saia um pouco para fora do card no desktop, criando um efeito visual interessante. Mas nos breakpoints menores essa imagem é reposicionada para dentro do fluxo normal, evitando problemas de elementos cortados ou scroll horizontal indesejado em telas pequenas.

As media queries funcionam em cascata, ou seja, as regras de um breakpoint maior são herdadas pelos menores a menos que sejam explicitamente sobrescritas.

Como funciona a criptografia de senha:

As senhas são protegidas usando SHA-256. Isso significa que, quando o usuário cria a senha, ela passa por uma transformação que a converte em um código irreversível. Quando armazenado no banco a senha real nunca fica armazenada. No login, a senha digitada passa pela mesma transformação e o sistema compara os códigos. Se forem iguais, o acesso é liberado. Isso torna o processo seguro, porque ninguém consegue voltar desse código para a senha original.

O SHA-256 é um algoritmo de hash que transforma qualquer dado de entrada em uma "impressão digital" única de 256 bits (uma sequência de 64 caracteres alfanuméricos). Ele funciona dividindo a entrada em blocos de 512 bits e

aplicando diversas operações lógicas e matemáticas ao longo de 64 ciclos de processamento.